FH University of Applied Sciences TECHNIKUM WIEN

Team Exercise

Identify, Refactor, and Improve Code Quality in a Mini-Module

Mag. Edin Pezerovic, MSc MSc

www.technikum-wien.at

FH University of Applied Sciences TECHNIKUM WIEN

Team Exercise

“Identify, Refactor, and Improve Code Quality in a Mini-Module”

Scenario

This exercise makes the team apply the principles from the attached document:

- Clean Code
- Code Smells
- SOLID
- DRY, KISS, YAGNI
- Readability
- Static Analysis
- Observability
- Code Review Practices
- Refactoring discipline

Starting Code

The starting code is provided in Java, C# and Python. You may use whichever of these languages you prefer for your solution.

Bad Python Version

```
# report_generator.py

import time

def generate_report(data, format):
    print("Starting report generation...")
    time.sleep(1)

    if format == "json":
```

```
            print("Generating JSON report...")
            result = "{"
            for key in data.keys():
                result += '"' + key + '": "' + str(data[key]) + '",'
            result = result[:-1] + "}"
            print(result)
            print("Sending report to monitoring system...")
            print("Report sent!")
            return result

        if format == "csv":
            print("Generating CSV report...")
            csv = ""
            for key in data.keys():
                csv += key + "," + str(data[key]) + "\n"
            print(csv)
            print("Sending report to monitoring system...")
            print("Report sent!")
            return csv

        print("Unknown format!")
        return None
```

Bad Java Version

```
    public class ReportGenerator {

        public static String generateReport(Map<String, Object> data, String
format) {
            System.out.println("Starting report generation...");
            try { Thread.sleep(1000); } catch (Exception e) {}

            if (format.equals("json")) {
                System.out.println("Generating JSON report...");
                String result = "{";
                for (String key : data.keySet()) {
                    result += "\"" + key + "\": \"" + data.get(key) + "\",";
                }
                result = result.substring(0, result.length() - 1) + "}";
                System.out.println(result);
                System.out.println("Sending report to monitoring system...");
                System.out.println("Report sent!");
                return result;
            }

            if (format.equals("csv")) {
                System.out.println("Generating CSV report...");
                String csv = "";
```

```
                for (String key : data.keySet()) {
                    csv += key + "," + data.get(key) + "\n";
                }
                System.out.println(csv);
                System.out.println("Sending report to monitoring system...");
                System.out.println("Report sent!");
                return csv;
            }

            System.out.println("Unknown format!");
            return null;
        }
    }
```

Bad C# Version

```
    using System;
    using System.Collections.Generic;
    using System.Threading;

    public class ReportGenerator
    {
        public static string GenerateReport(Dictionary<string, object> data,
string format)
        {
            Console.WriteLine("Starting report generation...");
            Thread.Sleep(1000);

            if (format == "json")
            {
                Console.WriteLine("Generating JSON report...");
                string result = "{";
                foreach (var entry in data)
                {
                    result += "\"" + entry.Key + "\": \"" + entry.Value +
"\",";
                }
                result = result.Substring(0, result.Length - 1) + "}";
                Console.WriteLine(result);
                Console.WriteLine("Sending report to monitoring system...");
                Console.WriteLine("Report sent!");
                return result;
            }

            if (format == "csv")
            {
                Console.WriteLine("Generating CSV report...");
                string csv = "";
```

```
                foreach (var entry in data)
                {
                    csv += entry.Key + "," + entry.Value + "\n";
                }
                Console.WriteLine(csv);
                Console.WriteLine("Sending report to monitoring system...");
                Console.WriteLine("Report sent!");
                return csv;
            }

            Console.WriteLine("Unknown format!");
            return null;
        }
    }
```

This code intentionally violates nearly every principle discussed.

Task Breakdown

This task is intended to be completed collaboratively by a group of at least two students.

Phase 1 — Joint Analysis

Together, identify 12–15 issues using the discussed principles.

Look for:

- Code smells (bloaters, couplers, dispensables, change preventers)
- SOLID violations (SRP, OCP, LSP, ISP, DIP)
- DRY / KISS / YAGNI violations
- Readability issues (naming, structure, clarity)
- Function design issues (too large, too many responsibilities)
- Class design issues (missing abstractions, no encapsulation)
- Static analysis warnings (duplication, complexity, string concatenation, magic values)
- Observability issues (poor logging, no structured output)

Deliverable: A bullet list of 12–15 concrete problems, each mapped to a principle discussed.

Phase 2 — Split Work

Student A — Architecture & Design

Create a redesign proposal applying the principles:

- Introduce SRP-aligned components (ReportFormatter, ReportSender, ReportGenerator).
- Apply OCP by making formats extensible without modifying core logic.
- Apply ISP by defining small, purpose-specific interfaces.
- Apply DIP by depending on abstractions (Formatter, Sender).
- Apply DRY by removing duplicated “send report” logic.
- Apply KISS by simplifying flow and removing unnecessary branching.
- Apply readability principles (intent-revealing names, small functions).
- Apply observability (structured logging, error handling).

Deliverable: A short architecture sketch (bullet points or diagram).

Student B — Refactoring & Implementation

Refactor the code into a clean, principle-aligned design:

- Extract formatters into separate classes
- Extract sending logic into a separate component
- Remove duplication
- Remove global prints, replace with structured logging
- Improve naming
- Improve readability
- Add error handling
- Add extensibility points
- Make the module test-ready
- Ensure behavior is preserved (refactoring rule)

Deliverable: A rewritten module applying the principles.

Phase 3 — Integration & Review

Together:

- Review the refactored code using the Code Review Checklist from the attached document
- Run a static analysis tool (e.g., Pylint) and discuss findings
- Evaluate readability, naming, complexity, and SOLID alignment
- Discuss how the new design improves:
  - maintainability
  - evolvability
  - testability
  - observability
  - long-term quality

Deliverable: A joint summary of improvements (5–10 bullet points).

Phase 4 — Final Output

Create a single-page summary containing:

- The original problems
- The redesigned architecture
- The refactored code
- The craftsmanship improvements
- The static analysis results
- The integration benefits

This becomes the team's final deliverable.